Recorded Future®

AI-Powered Malware Analysis

Integrating Recorded Future, Ghidra, and MCP for Next-Generation Cyber Defense

Presented by: Dana Behling, Insikt Group

One-day hands-on class | October 16, 2026



© 2026 Recorded Future. All rights reserved. Internal use only. Do not distribute externally.

Internal use only. Do not distribute externally.

Transforming Malware Analysis in Cyber Defense

Modern cyber defense requires bridging the gap between dynamic threat intelligence and deep static analysis. By integrating Recorded Future's automated sandbox with Ghidra's reverse engineering suite via the Model Context Protocol (MCP), Security Operations Centers can achieve unprecedented efficiency.

Unified Analysis

Connect behavioral intelligence directly to code-level analysis.

AI Augmentation

Leverage MCP to allow AI assistants to reason across disparate toolsets.

Accelerated Triage

Reduce the manual overhead required to extract actionable indicators of compromise (IOCs).

Property of Recorded Future

Internal use only. Do not distribute externally.

The Malware Triage Challenge: Bottlenecks in SOC Workflows

Security Operations Centers face a critical bottleneck when triaging sophisticated malware. Analysts must constantly pivot between disparate tools, losing valuable context and time.

01

Volume vs. Capacity

The sheer volume of suspicious files exceeds the manual capacity of available reverse engineers.

02

Context Switching

Analysts lose efficiency when moving between threat intelligence portals, sandboxes, and disassemblers.

03

Siloed Data

Behavioral data from dynamic execution rarely maps automatically to the static code functions responsible for that behavior.

Property of Recorded Future

Internal use only. Do not distribute externally.

Class at a glance: six one-hour blocks

A hands-on day that starts in the Sandbox, moves into Ghidra with AI through MCP, then returns to Recorded Future Malware Intelligence.

H1	**Sandbox kickoff**	A behavior timeline, an initial evidence table and questions for code analysis
H2	**Grumpy Ferret: execution flow**	A labeled execution flow connected to Sandbox observations
H3	**Grumpy Ferret: configuration**	Decoded configuration, recovered serial logic and checked findings
H4	**Soggy Badger**	A verified stack string, the hash-check approach and one host artifact or anti-debug check
H5	**Restless Otter**	Decoded runtime configuration, a verified anti-debug check and the ticket-to-decryption link
H6	**Malware Intelligence and wrap-up**	Intelligence pivots from verified findings and a concise analysis handoff

Each block is 60 minutes including Q&A and a short break. Lunch is additional.

Property of Recorded Future

Internal use only. Do not distribute externally.

How we will work today

No reverse-engineering experience needed. The audience works in SOC triage, so every technical term is explained in plain language.

01

Sandbox first

Start with what the files do when they run: processes, DNS lookups, HTTP requests, files and registry changes. Capture questions for deeper analysis.

02

AI through MCP

Use an AI assistant connected to the open Ghidra project to investigate the code behind those observations. Ask for addresses and references.

03

Check every answer

AI output is a lead until it is checked against the decompiler, the references and the underlying data.

GOAL By the end you can explain a file's behavior and support it with Sandbox, code and intelligence evidence.

Property of Recorded Future

Internal use only. Do not distribute externally.

Meet the lab files

Three core exercises plus one bonus. Students receive the release binaries, not the source code.

LINUX ELF

Grumpy Ferret

Hours 2 and 3

XOR configuration, DNS and HTTP activity, anti-debugging, decoy strings and a reversible serial check.

WINDOWS PE

Soggy Badger

Hour 4

Stack strings, a named mutex, a temporary registry marker, API and PEB anti-debugging and a ROR13 hash check.

MACOS MACH-O

Restless Otter

Hour 5

Encrypted strings and configuration, sysctl anti-debugging and a transformed ticket with a checksum.

BONUS

Vexed Wolverine

Early finishers or take-home

An expert reserve challenge, outside the planned completion path for the class.

Property of Recorded Future

Internal use only. Do not distribute externally.

Threat Intelligence & Built-in Sandbox

Comprehensive Intelligence

Recorded Future delivers real-time context on emerging threats, adversaries, and infrastructure to empower proactive defense.

Key Takeaway

Seamlessly pivot from high-level threat actor profiling to granular file-level analysis using the integrated detonation environment.

Sandbox Capabilities

Automated Detonation

Safely execute suspicious files in a controlled, instrumented environment.

Network Profiling

Capture command and control (C2) communications, DNS requests, and payload downloads.

System Artifacts

Identify dropped files, registry modifications, and process injections mapped to MITRE ATT&CK tactics.

Property of Recorded Future

Internal use only. Do not distribute externally.

Automated Sandbox Triage

Behavior Analysis and IOC Extraction

Automated triage transforms raw execution data into structured, actionable intelligence. The sandbox automatically parses the execution chain to surface the most critical artifacts for defenders.

Extracted Intelligence Categories

Network IOCs

- **Malicious IP addresses**
- Domains involved in C2 infrastructure
- Specific URI patterns extracted from network traffic

Host IOCs

- **File hashes (SHA-256)**
- Unique system mutexes created
- Modified or created registry keys

Behavioral Signatures

Automated mapping

- Maps API calls directly to known malicious techniques

Immediate Risk Scoring

- Calculates actionable risk scores for analyzed samples

Property of Recorded Future

Internal use only. Do not distribute externally.

Understanding Model Context Protocol (MCP)

The **Model Context Protocol (MCP)** is an open standard designed to connect AI models securely to external data sources and developer tools.

It acts as the **critical bridge** in modern security workflows.

Key Mechanisms

01

Standardized Integration

Provides a universal API for AI assistants to query databases and interact with local applications.

02

Context Enrichment

Allows Large Language Models (LLMs) to read analysis reports and source code simultaneously.

03

Secure Execution

Ensures AI agents interact with sensitive security tools within strictly defined boundaries and permissions.

Internal use only. Do not distribute externally.

Static Analysis with Ghidra: Reverse Engineering

Ghidra, the software reverse engineering (SRE) suite developed by the NSA, provides deep visibility into the internal mechanics of compiled malware.

CORE CAPABILITIES

01

Advanced Decompilation

Translates complex machine code and assembly language back into readable C-like pseudocode.

02

Architecture Support

Handles a vast array of processor architectures (x86, ARM, MIPS) commonly targeted by malware.

03

Collaborative Analysis

Enables multiple analysts to work on the same binary simultaneously, annotating functions and data structures.

Property of Recorded Future

Internal use only. Do not distribute externally.

Integrating Sandbox Intel into Ghidra via MCP

By leveraging MCP, we can overlay dynamic sandbox intelligence directly onto static code analysis, dramatically accelerating the reverse engineering process.

STEP 01

Automated Annotation

AI utilizes MCP to read Recorded Future sandbox reports and automatically comment Ghidra decompilation with behavioral context.

STEP 02

Function Mapping

Network callbacks observed in the sandbox are algorithmically linked to the specific network API calls in Ghidra.

STEP 03

Guided Reversing

The AI assistant directs analysts to the most critical functions (e.g., encryption routines or anti-analysis checks) based on dynamic execution results.

Property of Recorded Future

Internal use only. Do not distribute externally.

Key terms: tools and evidence

Plain-language meanings for words you will hear all day. No reverse-engineering experience needed.

Sandbox

An isolated environment that runs a file and records processes, DNS, HTTP, files and registry changes.

Ghidra

A free reverse-engineering tool. Its decompiler rewrites machine code as C-like pseudocode you can read.

MCP

The connector that lets the AI assistant query the open Ghidra project. Its output is a lead until checked.

Function address

Where a function sits in the program, written in hex, such as 0x00136bda.

Cross-reference (xref)

A record of where a function or data item is called or used. Show the callers means list those places.

Checkpoint

A saved Ghidra project state with labels and notes, so a group can start from a known point.

Property of Recorded Future

Internal use only. Do not distribute externally.

Key terms: reading the code

Plain-language meanings for words you will hear all day. No reverse-engineering experience needed.

Machine code

The raw CPU instructions in a program, shown in assembly-like form called disassembly.

Argument, consumer

An argument is a value passed into a function. The consumer is the code that uses it afterward.

Configuration

Settings a sample stores internally, such as a server name. Often hidden with simple encoding.

XOR and repeating key

A reversible scramble: applying the same key twice restores the byte. A repeating key cycles through key bytes.

Stack string

Text built one character at a time in memory, so a plain strings scan does not find it.

Anti-debugging

Code that checks whether a debugger is attached and changes behavior if one is.

Property of Recorded Future

Internal use only. Do not distribute externally.

Key terms: intelligence and evidence

Plain-language meanings for words you will hear all day. No reverse-engineering experience needed.

Indicator

A searchable piece of evidence: a domain, IP address, file hash or string.

Pivot

Moving from one indicator to related infrastructure, files or reports.

Entity

Something Recorded Future tracks: a domain, IP, file hash, malware family or threat actor.

No match

The platform has no record for that query. It does not show the value is safe.

Attribution

Linking activity to a named actor or family. Shared infrastructure alone does not prove it.

Fixture and decoy

A fixture is made-up input standing in for a live service, labeled synthetic. A decoy is a fake value meant to mislead.

Property of Recorded Future

Internal use only. Do not distribute externally.

Five statuses for every finding

Record evidence source, analytical status and operation outcome separately. One label cannot describe all three.

01	**Observed**	Directly recorded in an identified Sandbox run or runtime capture. Name the environment, report reference and date.
02	**Attempted**	Evidence shows an operation started. Record the outcome separately. A DNS request with no answer is an attempt, not a confirmed connection.
03	**Decoded**	Output reproduced from bytes and a decoder recovered from code, without running the sample. Cite the data and decoder locations.
04	**Inferred**	An interpretation supported by identified code or other evidence but not reproduced or observed. State the basis and the uncertainty.
05	**Unverified**	A source or AI claim whose supporting evidence has not been checked yet.

Fixture results carry a synthetic fixture tag. They never establish a live request or download.

Property of Recorded Future

Internal use only. Do not distribute externally.

One evidence template for the whole class

Printed as a one-page handout at the start of Hour 1. Fill one row for every finding, in every block.

Finding or value

The exact value or behavior you record.

Sample and role

The exact sample and the role the value plays.

Source and date

Runtime, code, intelligence or synthetic fixture, with date.

Reference or address

A report event or function address another analyst can find.

Status

Observed, attempted, decoded, inferred or unverified.

Operation outcome

Succeeded, failed, timed out, unknown or not applicable.

Confidence and reason

Why you trust it. A checked call site counts; fluent AI prose does not.

Next question

The specific question that is still unresolved.

Property of Recorded Future

Internal use only. Do not distribute externally.

Paired practice: operator and reviewer

Ten-minute tasks sit inside each block. They do not extend the six-hour schedule.

01

Partner A operates

Runs the tools: the Sandbox report, Ghidra and the AI assistant. With one working MCP connection, share that station. Without one, inspect the prepared project by hand.

02

Partner B challenges

Questions every claim, asks for the address or event behind it and writes the evidence row. Switch roles after five minutes.

03

Stop at ten minutes

Each pair reports what it established and what is still open. Slower pairs join from the prepared checkpoint. Early finishers take the stretch goal.

Property of Recorded Future

Internal use only. Do not distribute externally.

HOUR 1

Recorded Future Sandbox kickoff

We begin with behavior, so students carry concrete evidence and questions into Ghidra.

STUDENTS PRODUCE

A behavior timeline, an initial evidence table and questions for code analysis

Property of Recorded Future

Internal use only. Do not distribute externally.

Hour 1 plan: from reports to questions

Prepared reports keep the class moving while any live submission runs.

10 min	**Goals and Sandbox orientation** What a report can establish, how to read execution settings and what a run did or did not test.
20 min	**Grumpy Ferret walkthrough** Process activity, signatures, DNS and HTTP observations and host artifacts, plus submission settings.
15 min	**Windows and macOS reports** Compare the prepared Soggy Badger and Restless Otter evidence and note platform limits.
10 min	**Draft a behavior timeline** Separate observed activity from attempted lookups and unverified leads.
5 min	**Questions and a short break**

STUDENTS PRODUCE

A behavior timeline, an evidence table and questions for static analysis.

PAIRED TASK

Pick a DNS event and a process or HTTP event. Output: two evidence rows and one question for code analysis.

Property of Recorded Future

Internal use only. Do not distribute externally.

Reading a Sandbox report

A report shows what happened in one run. Be clear about what it can and cannot establish.

A report can show

- Processes and child processes that ran
- DNS lookups and HTTP requests that were attempted
- Signatures, files and registry changes
- The execution settings and environment used

It cannot show on its own

- Code paths that did not run in this environment
- That a requested domain is malicious
- Whether a connection succeeded, unless the outcome is recorded
- Behavior on platforms it does not run, such as a prepared macOS capture

REMEMBER A requested domain, an attempted connection or a host artifact is not automatically a malicious indicator.

Property of Recorded Future

Internal use only. Do not distribute externally.

Grumpy Ferret in the Sandbox

Record only what the report shows. Code explains the rest later.

1	**Process activity**	The main process and the child process that makes the web requests.
2	**DNS and HTTP**	Requests to YouTube and GitHub, with the outcome of each recorded separately.
3	**Host artifacts**	Files or other artifacts the report lists, with the settings and arguments used.

HOLD THAT THOUGHT Do not decode the Python stage here. It is revealed in Hour 3. If the classroom network blocks the requests, we use an instructor fixture and label it synthetic.

Property of Recorded Future

Internal use only. Do not distribute externally.

HOUR 2

Grumpy Ferret: orientation and execution flow

We start the Ghidra portion by tracing the code behind what the Sandbox observed.

STUDENTS PRODUCE

A labeled Grumpy Ferret execution flow connected to observed behavior

Property of Recorded Future

Internal use only. Do not distribute externally.

Hour 2 plan: tracing behavior to code

A guided walkthrough with focused AI prompts. Verify everything in Ghidra.

10 min — **Import and orient**
Open the prepared project and meet the decompiler, references and Ghidra MCP tools.

25 min — **Focused AI prompts**
Find the main path and the DNS and HTTP routines. Ask for addresses, verify, compare with the timeline.

20 min — **Error exercise and tracing**
Five minutes on a deliberate AI error, ten on paired tracing, five to apply checked names and comments.

5 min — **Questions and a short break**

STUDENTS PRODUCE

A labeled execution flow connected to observed behavior.

PAIRED TASK

Trace one Sandbox network observation to an API call and the origin of its argument. Output: event reference, call-site address and argument source.

Property of Recorded Future

Internal use only. Do not distribute externally.

The AI and MCP method

A repeatable loop for every Ghidra block. The assistant proposes, you verify.

STEP 01

Ask a focused question

Name one thing to find, such as the code that makes a DNS request.

STEP 02

Demand addresses

Ask for function addresses, callers and supporting references.

STEP 03

Verify in Ghidra

Open each cited address. Read the decompiler output, references and data.

STEP 04

Annotate checked facts

Apply names and comments only after the evidence supports them.

EXAMPLE PROMPT

Confirm the active program and map its main execution path. Locate the routines responsible for our observed DNS and HTTP activity, show callers and argument origins, and cite addresses. Distinguish verified roles from hypotheses. Do not rename, patch or run the file.

Property of Recorded Future

Internal use only. Do not distribute externally.

Deliberate AI error: check before you rename

A five-minute exercise. The answer below was written by the instructor to sound confident while citing nothing checkable.

INSTRUCTOR-WRITTEN FLAWED AI-STYLE ANSWER

The flag-like string found by the strings scan is the genuine success flag. Rename its referencing function reveal_real_flag.

A string can be unused or sit on a failure or anti-debug path. Showing up in a strings scan does not make it the success reveal.

1	**Trace references**	Find every reference to the string.
2	**Read the branches**	Read the conditions that use it.
3	**Find the real path**	Locate the separate validation and success path.
4	**Rewrite the claim**	One sentence with a checked address.

Student output: one corrected sentence with a reference. If the string is unused, report that and give it no runtime role.

Debrief: which evidence disproved the claim, and which annotation would have been misleading?

Property of Recorded Future

Internal use only. Do not distribute externally.

HOUR 3

Grumpy Ferret: configuration and validation

We focus on the functions that explain what the Sandbox could not show directly.

STUDENTS PRODUCE

An evidence-backed explanation of Grumpy Ferret that separates traffic, decoded configuration and strings

Property of Recorded Future

Internal use only. Do not distribute externally.

Hour 3 plan: configuration and validation

Prepared project and guided prompts. Label every finding by where it came from.

25 min

Configuration recovery

Fifteen minutes of demonstration, then ten in pairs to reproduce one decoded field and find its consumer.

15 min

Serial check and gates

Explain the reversible serial transform. Find the anti-debug gate and decoys and note what runs first.

15 min

Consolidate findings

Label each finding: observed in Sandbox, recovered from code or only a string. Flag candidates for Hour 6.

5 min

Questions and a short break

STUDENTS PRODUCE

A checked explanation of Grumpy Ferret and an evidence table that separates traffic, decoded configuration and strings.

PAIRED TASK

Reproduce one configuration field and follow it into a consumer. Output: decoder and data references, the recovered field and the consuming call.

Property of Recorded Future

Internal use only. Do not distribute externally.

Reversible math in plain language

Four building blocks that return in Hours 3, 4 and 5. If you can read these, you can follow most decoders.

XOR

A reversible scramble. Applying the same key a second time restores the original byte.

A XOR K = C, then C XOR K = A

Repeating key

A short key cycles across a run of bytes. Byte i uses key byte i mod length.

3-byte key: 0, 1, 2, 0, 1, 2 ...

mod 256 and mod 2^32

Keep a number inside one byte (0 to 255) or one 32-bit value, like an odometer rolling over.

250 + 10 = 4 (mod 256)

Rotate right (ROR)

Shift bits right and wrap the bits that fall off back onto the left. ROR13 rotates by 13 bits.

1011 rotated right by 1 = 1101

Property of Recorded Future

Internal use only. Do not distribute externally.

Grumpy Ferret: what the code explains

These functions explain what the Sandbox could not show. Keep exact inputs and flags out of shared summaries.

01

Configuration decoder

Recover the data location, byte count, key and index rule. Reproduce the output without running the file, then trace one field into its consumer.

02

Serial check

A reversible transform checks the serial a user types in. Recover it, then validate any candidate by reapplying the original transform.

03

Anti-debug gate

Find the TracerPid branch. Establish which calls happen before it and which behavior it prevents. Decoys look important but lead nowhere.

RULE Label each finding: observed in Sandbox, recovered from code, or only present as a string.

Property of Recorded Future

Internal use only. Do not distribute externally.

Four questions to demonstrate

Each question asks for evidence, not just an answer.

1	**Find the code**	Find the function responsible for this DNS request and show the supporting references.
2	**Explain a decoder**	Explain this decoder and reproduce its output without executing the binary.
3	**Check the order**	Which behavior occurred before the anti-debug check, and which behavior does the check prevent?
4	**Separate fact from guess**	Summarize what is confirmed, what is inferred and what we still don't know.

Property of Recorded Future

Internal use only. Do not distribute externally.

HOUR 4

Soggy Badger in Ghidra

We apply the same AI and MCP method to a Windows program, using the Sandbox report as a reference.

STUDENTS PRODUCE

A verified stack string, the hash-check approach and one host artifact or anti-debug check

Property of Recorded Future

Internal use only. Do not distribute externally.

Hour 4 plan: Soggy Badger on Windows

A prepared project with selected goals. Not every path needs to be reversed.

10 min	**Windows report and PE imports**	Revisit key Sandbox findings, inspect imports and find the main path with AI and MCP.
20 min	**Stack strings**	Ten minutes of demonstration, then ten in pairs to recover a string and trace its consumer.
15 min	**The ROR13 hash check**	Explain the hash routine and test candidate imported DLL names with the recovered algorithm.
10 min	**Host artifact or anti-debug**	Find one anti-debug check (API or PEB) and the registry check-in or cleanup routine.
5 min	**Questions and a short break**	

STUDENTS PRODUCE

A verified stack-built string, the hash-check approach and one host artifact or anti-debug check.

PAIRED TASK

Reconstruct a selected stack string and explain where it is used. Output: bytes ordered by buffer offset, the builder address and the consumer.

Property of Recorded Future

Internal use only. Do not distribute externally.

Stack strings: text built in pieces

A stack string is text the program assembles one character at a time in memory, so a plain strings scan misses it.



The bytes are written one at a time, often out of order and sometimes several at once. They never sit together in the file for a strings scan to find.

Order by destination offset

Sort the writes by where they land in the buffer, not by the order of the instructions.

Watch widths and byte order

A single store can write several bytes at once. Byte order decides which character lands first.

Check the end and the use

Confirm the terminator, then trace the consumer to see what the string is used for.

Property of Recorded Future

Internal use only. Do not distribute externally.

The ROR13 hash check

The program hashes an input string and compares the result with a fixed target. The hash is one-way, so we test candidates.

START

h = 0

FOR EACH CHARACTER

Rotate h right by 13 bits, then add the character

AT THE END

Compare the final h with a fixed target

h = (ROR32(h, 13) + input_byte) mod 2^32

Test candidates, do not reverse

Hash candidate imported DLL names with the recovered algorithm and see which one reproduces the target.

Use exact bytes and 32-bit math

Use uppercase DLL base names without the .DLL suffix. Do not assume automatic case normalization or a unique inversion.

Property of Recorded Future

Internal use only. Do not distribute externally.

Windows host artifacts and anti-debugging

Compare the code with the Windows Sandbox report. Do not assume every code path ran.

01

Named mutex

A named marker that stops a second copy of the program from running.

02

Registry marker

A registry value written as a host artifact and later removed. A cleanup path shows intended lifecycle. Claim cleanup was observed only if the report shows the event.

03

API and PEB checks

An API call or a read of the PEB, the Windows process block with a being-debugged flag. Find one gate and the condition that controls it.

STRETCH Trace every registry, networking and anti-debugging path. Skip this if the block runs long.

Property of Recorded Future

Internal use only. Do not distribute externally.

HOUR 5

Restless Otter in Ghidra

We apply the workflow to a macOS Mach-O, using a prepared checkpoint and staged hints.

STUDENTS PRODUCE

Decoded runtime configuration, a verified anti-debug check and the ticket-to-decryption relationship

Property of Recorded Future

Internal use only. Do not distribute externally.

Hour 5 plan: Restless Otter on macOS

A prepared checkpoint with staged hints. Compare with the Linux and Windows examples.

10 min	**Mach-O and the prepared project** Find the main flow with AI and MCP. Use the macOS capture; Sandbox macOS support is not assumed.
25 min	**Decoder and sysctl gate** Fifteen minutes locating the decoder and gate, then ten in pairs on configuration and a consumer.
10 min	**Ticket demonstration** The instructor shows the 16-character ticket, its transform and its link to flag decryption.
10 min	**Compare the three crackmes** String hiding, validation and anti-debugging across Linux, Windows and macOS.
5 min	**Q&A and take-home next steps**

STUDENTS PRODUCE

Decoded configuration, a verified sysctl gate and a comparison of techniques across the core crackmes.

PAIRED TASK

Recover two configuration fields and verify one consumer or the gate. Output: decoded fields and addresses, with runtime status kept.

Property of Recorded Future

Internal use only. Do not distribute externally.

Three executable formats

The workflow stays the same while the platform details change.

PE

Windows executables

The import list shows which DLLs and APIs the program calls.

Soggy Badger

Hour 4

ELF

Linux executables

Checks such as TracerPid read a field from /proc/self/status.

Grumpy Ferret

Hours 2 and 3

MACH-O

macOS executables

Restless Otter is arm64 only, so it will not run on Intel Macs.

Restless Otter

Hour 5

NOTE Sandbox macOS support is not assumed. We use a prepared macOS capture and static analysis, and say so when we cite it.

Property of Recorded Future

Internal use only. Do not distribute externally.

Restless Otter: decoder, gate and ticket

Staged hints and a prepared checkpoint keep the hour on track.

01

String decoder

A shared routine turns scrambled bytes back into readable text. Find its callers, recover two configuration fields and check one consumer.

02

sysctl anti-debug gate

macOS asks the kernel whether a debugger is attached through the P_TRACED flag. Show how the branch gates the reveal path.

03

The ticket (demo)

A 16-character ticket is scrambled per character and fed into a rolling checksum. A validated ticket feeds flag decryption. Students verify length and comparison.

EXTENSION Take-home: recover the ticket yourself, reproduce the full checksum and trace the complete flag-key derivation.

Property of Recorded Future

Internal use only. Do not distribute externally.

Comparing the three core crackmes

Same questions, different platforms. For each finding, ask whether it was observed or derived from code.

Technique	Grumpy Ferret (Linux)	Soggy Badger (Windows)	Restless Otter (macOS)
String hiding	XOR configuration with a repeating key	Stack strings built one character at a time	Encrypted strings and configuration, one shared decoder
Validation	Reversible serial transform	ROR13 hash of the input compared with a constant	16-character ticket with a rolling checksum
Anti-debugging	TracerPid in /proc/self/status	API check and a PEB flag	sysctl and the P_TRACED flag
Runtime evidence	Sandbox report	Windows Sandbox report	Prepared macOS capture

Property of Recorded Future

Internal use only. Do not distribute externally.

HOUR 6

Malware Intelligence follow-up and wrap-up

We return to Recorded Future with verified evidence from all three core crackmes.

STUDENTS PRODUCE

Intelligence pivots from verified findings and a concise, evidence-based analysis handoff

Property of Recorded Future

Internal use only. Do not distribute externally.

Hour 6 plan: back to Malware Intelligence

Evidence-led searches and pivots, then a handoff that separates what we know by source.

10 min — **Qualify the evidence**
Revisit the evidence table. Pick verified findings and label the source and confidence.

25 min — **Lookups and a pivot**
Five minutes demonstrating a lookup, ten on paired searches and ten on a prepared real-world pivot.

15 min — **Analysis handoff**
Separate Sandbox observations, Ghidra findings and intelligence context. Note gaps and confidence.

5 min — **Follow-up question**
One evidence-backed follow-up question or hunting hypothesis.

5 min — **Q&A and take-home next steps**

STUDENTS PRODUCE

Intelligence pivots from verified findings and a concise handoff.

PAIRED TASK

Look up one recovered network value and one contrasting artifact. Output: exact queries, actual outcomes, references and a revised interpretation.

Property of Recorded Future

Internal use only. Do not distribute externally.

Lookups that keep the code role

Students choose two values from their own verified evidence and record actual results. These are the rehearsal values.

Value	Role found in code	What to conclude
api.ipify.org	Grumpy Ferret public IP check	Shared-service context needs the code role. The domain alone does not attribute the sample.
example.com	Classroom DNS and HTTP target	Explain the exercise use. Keep platform results without turning benign activity into threat evidence.
203.0.113.7	Embedded documentation address	A placeholder can be recovered accurately and still offer no operational pivot.
c2.grumpyferret-demo.invalid	Classroom probe or decoy configuration	Record accepted, no match or unsupported accurately and explain the training role.
SHA-256 of the release binary	Exact sample identity	Use the computed hash. No record found does not establish safety.

Property of Recorded Future

Internal use only. Do not distribute externally.

Reading lookup results

Benign training files may have no threat records. The goal is evidence-led searching, not a hit.

01

No record found

The platform has no record. That does not establish safety or erase behavior already established.

02

Unsupported or failed

Unsupported, access denied and request failure are different outcomes. Record which one you got.

03

Shared infrastructure

A shared host or IP alone does not establish actor or malware family attribution.

04

Context is not proof

Platform context about a shared service says little about our sample without the code role.

Property of Recorded Future

Internal use only. Do not distribute externally.

A defensible pivot

An instructor demonstration on a real-world record chosen before class. It is separate from the crackmes.

1 Selected indicator → 2 Originating report → 3 Related entity or report → 4 Evidence and dates → 5 Follow-up question

01

Open the evidence

Open the relationship's evidence and ask students what it supports.

02

Test the timeframe

Check publication and observation dates. Discuss alternative explanations.

03

Guard attribution

A shared host or IP alone does not establish actor or malware family attribution.

Property of Recorded Future

Internal use only. Do not distribute externally.

The analysis handoff

Four sentences that tie the opening question to runtime evidence, code findings and intelligence.

1	**What we knew first**	The first assessment from the Sandbox timeline and the opening question.
2	**What code and runtime added**	Verified behavior and code findings, each with an address or event reference.
3	**What intelligence added or lacked**	A sourced finding, a supported relationship or a justified limit, including no match.
4	**The next step**	One evidence-backed follow-up question or hunting hypothesis.

PARTNER CHECK Include sample identity and limits. Your partner checks one code reference and one intelligence reference.

Property of Recorded Future

Internal use only. Do not distribute externally.

Closing the loop: debrief

Five questions to end the class. Strong answers cite an address, an event or a dated source.

Q1	**What changed after code analysis?**	A supported role or branch condition, with its address.
Q2	**What did intelligence add?**	A sourced finding, a supported relationship or a justified limit.
Q3	**Does no match mean safe?**	No. Missing intelligence is not the same as behavior already established.
Q4	**What makes a pivot defensible?**	A cited relationship relevant to the question and timeframe, with uncertainty kept.
Q5	**What closes the loop?**	A revised assessment tied to the opening question, runtime evidence, code findings and a next action.

Property of Recorded Future

Internal use only. Do not distribute externally.

EXTENSION

After the core hours: from findings to detections

YARA and Sigma are an extension, not part of the six core hours.

KEEP IN MIND

Automated annotation, function mapping and rule generation are demonstrated capabilities with analyst verification

Property of Recorded Future

Internal use only. Do not distribute externally.

Automated Rule Generation: YARA and Sigma

The ultimate goal of malware analysis is generating robust detections. The AI-driven pipeline automatically translates combined static and dynamic findings into deployable engineering rules.

STATIC ANALYSIS

YARA Rules

Generated based on unique string combinations, byte sequences, and specific function structures identified in Ghidra.

DYNAMIC ANALYSIS

Sigma Rules

Created from the behavioral logs and process execution chains captured during Recorded Future sandbox detonation.

Continuous Refinement

AI evaluates generated rules against known goodware to minimize false positive rates before deployment.

Property of Recorded Future

Internal use only. Do not distribute externally.

Enhancing Threat Hunting Across the Enterprise

With high-fidelity rules and extracted IOCs in hand, threat hunters can proactively sweep the enterprise environment to identify latent compromises.

01	**SIEM Integration**	Deploy Sigma rules to the Security Information and Event Management system to detect historical behavioral patterns.
02	**EDR Sweeps**	Push YARA rules to Endpoint Detection and Response platforms to scan memory and disk for the identified malware variants.
03	**Intelligence Sharing**	Export validated threat artifacts back into the Recorded Future platform to enrich the global threat landscape.

Internal use only. Do not distribute externally.

Streamlining Analyst Workflows & Reducing MTTR

Implementing an AI-powered, MCP-bridged analysis pipeline fundamentally shifts SOC metrics, moving teams from reactive alert fatigue to proactive defense.

EFFICIENCY

Reduced MTTR

Mean Time to Respond drops significantly as manual context switching and basic reverse engineering tasks are automated.

SCALABILITY

Skill Scalability

Junior analysts can leverage AI-guided insights to perform complex triage previously requiring senior reverse engineers.

EXPERTISE

Focused Expertise

Senior analysts are freed from routine extraction tasks, allowing them to focus on novel, highly sophisticated threats.

Property of Recorded Future

Internal use only. Do not distribute externally.

Bonus challenge: Vexed Wolverine

For early finishers and take-home. It is not part of the planned completion path for the class.

01

The challenge

Recover a unique 32-character ticket and prove you have the genuine flag.

02

Why it is hard

Encrypted configuration and bytecode, two virtual machines, shuffled opcodes, masked operands, flattened control flow, integrity checks, anti-debug checks and decoys.

03

How to approach it

Document the validation path and verify AI findings in Ghidra. Recovering one decoder or interpreter is useful progress, even without a finish.

SAFE No brute-forcing is needed. The challenge has no network activity, persistence, injection or credential collection.

Property of Recorded Future

Internal use only. Do not distribute externally.

Conclusion & Roadmap: Building an Integrated Pipeline

The integration of Recorded Future, Ghidra, and MCP represents a paradigm shift in malware analysis, transforming siloed tools into a cohesive, AI-driven defense mechanism.

Next Steps:

01

Pipeline Deployment

Begin integrating MCP endpoints within your existing sandbox and reverse engineering environments.

02

Custom AI Prompts

Develop tailored system prompts for your AI assistants to standardize how they interpret sandbox reports.

03

Automation Expansion

Continuously expand the automated rule generation capabilities to cover emerging detection frameworks.

Property of Recorded Future

Internal use only. Do not distribute externally.